



Analyzing Transaction Graphs via Motif-Based Graph Representation Learning for Cryptocurrency Price Prediction

Peker Celik¹ · Emre Sefer¹ 

Accepted: 25 March 2025 / Published online: 18 April 2025
© The Author(s) 2025

Abstract

Decentralized and transparent nature of cryptocurrencies have lately increased investors interest in them. Forecasting cryptocurrency's price accurately is crucial to come up with a good investment strategy, and such a forecast requires one to consider its unique attributes as well as high volatility. Even though many existing studies have focused on analyzing the cryptocurrency transaction graph topology, studies on the analysis of transaction graph's impact on prices are quite limited. In this paper, we explore the forecasting ability of blockchain transaction graph-based attributes on Bitcoin's and Ethereum's future price via deep learning methods. More specifically, we came up with motif convolution module (MCM), a motif-based graph representation learning approach to take local structural knowledge into account more strongly in node and edge-attributed transaction graphs encoding substantial structural knowledge. Our proposed MCM constructs a motif dictionary without supervision, and employs a new motif convolution operation while extracting the vertices local structural context. Afterwards, we learn high-level vertex embeddings by using such structural context via multilayer perceptron and graph neural network. Overall, we extract the attributed transaction graphs temporally-evolving low-dimensional representations, and use such embedding data together with historical prices within self-attention-based LSTM to predict the future prices accurately. Our proposed approach outperforms all considered baselines in terms of both price and price direction prediction, showing the promise of efficient integration of transaction data into cryptocurrency price prediction.

Keywords Graph neural network · Deep learning · Cryptocurrency · Price prediction

✉ Emre Sefer
emre.sefer@ozyegin.edu.tr

Peker Celik
peker.celik@ozu.edu.tr

¹ Artificial Intelligence and Data Engineering Department, Ozyegin University, Orman Sokak, 34794 Cekmekoy, Istanbul, Türkiye

Abbreviations

MCM	Motif convolution module
BTC	Bitcoin
ARG	Attributed relational graph
MCL	Motif convolution layer
MLP	Multilayer perceptron
QAP	Quadratic assignment problem
CTA	Cooperative thread array
UTXO	Unspent transaction output
MAE	Mean absolute error
MSE	Mean squared error
RMSE	Root mean squared error
MAPE	Mean absolute percentage error

1 Introduction

World countries financial systems are turning into more digital systems extraordinarily fast (Huawei & Oxford Economics, 2017; Patro et al., 2011; Sefer, 2022a). Cryptocurrencies have been created from an open, anti-forgery registry framework called blockchain. Such cryptocurrency breaks away from conventional trust relationships, and it does not rely on any central authority. Since cryptocurrency is decentralized and transparent, it introduces a possible monetary system that protects the privacy (Herskind et al., 2020; Sefer, 2022b). Bitcoin, which can be seen as the first cryptocurrency, came out in 2009 with the Genesis Block generation. Afterwards, alternative cryptocurrencies started to appear. Among those alternative cryptocurrencies, Ethereum has the highest popularity. Ethereum allows the transmission of money in a decentralized way, as well as makes it possible to participate in rule-based smart contracts.

Volatility of price is one of the principal issues of cryptocurrencies. Bitcoin (BTC) prices have been extremely volatile since 2013. For instance, its price has skyrocketed by almost 1900% in 2017, while it has decreased by almost 72% in 2018. Even though Bitcoin prices are extraordinarily volatile, they can still recover even when the market uncertainty is quite high such as during the COVID-19 pandemic (Mudassir et al., 2020). Volatile Bitcoin prices depend not only on market's supply and demand, but also on various other cryptocurrency-specific factors such as mining cost, competing cryptocurrencies, number of transactions in the blockchain, etc.

Cryptocurrency evaluation can be divided into fundamental, technical, or quantitative analysis. Technical analysis in Bitcoin trading is commonly used as a directional bet on the market. As technical indicators fluctuate quite frequently for cryptocurrencies, anticipating the cryptocurrency price via technical analysis is demanding. To handle those difficulties, several studies have combined machine learning algorithms with technical indicators to predict particularly Bitcoin price (Huang et al., 2019). On the other hand, a number of studies have instead focused on quantitative forecasting of Bitcoin price. Greaves and Au (2015) generates a

Bitcoin transaction graph, and forecasts Bitcoin price in an hourly interval by using the knowledge in transaction graph via machine learning approaches. A number of other studies predict Bitcoin price by using blockchain data, such as difficulty, hash rate, and transaction volume (Jang & Lee, 2018; Jay et al., 2020; Saad & Mohaisen, 2018). Lastly, some studies such as Indera et al. (2017) combine technical and quantitative analysis where they construct a Multi-Layer Perceptron-based approach that uses open, close, minimum, and maximum historical prices combined with moving average technical indicators.

The current studies above have focused on predicting mainly Bitcoin and Ethereum prices via machine learning approaches, but without entirely utilizing the attributes and dynamics of publicly available transaction graph and associated on-chain data which are unique to cryptocurrencies and do not exist for other more traditional assets. The number of recent studies that use transaction graph qualities and on-chain data features on price forecasting is limited (Gezici & Sefer, 2024; Greaves & Au, 2015; Jagannath et al., 2021a, 2021b; Kim et al., 2022; Sefer, 2021). Such crypto-specific data introduces unexplored ways to realize the knowledge in transaction data which has previously been overlooked, as well as utilize more modern methods to model the cryptocurrency-associated volatility more accurately.

Here, we explore the forecasting ability of blockchain transaction graph-based attributes on Bitcoin's and Ethereum's future prices via deep learning. More specifically, we came up with motif convolution module (MCM), a motif-based graph representation learning approach to take local structural knowledge into account more strongly in node and edge-attributed transaction graphs encoding substantial structural knowledge. Here, we assume cryptocurrency transaction graphs will be modeled as attributed relational graphs (ARGs). This ARG characterization enhances normal graph representation by relating features with vertices and links to better explain the entities and relations between them. Our proposed MCM constructs a motif dictionary without supervision, and employs a new motif convolution operation while extracting the vertices local structural context. Afterwards, we learn high-level vertex embeddings by using such structural context via multilayer perceptron and graph neural network. Overall, we extract the attributed transaction graphs temporally-evolving low-dimensional representations, and use such embedding data together with historical prices within self-attention-based LSTM to predict the future prices accurately. Our approach outperforms all considered baselines in terms of both price and price direction prediction, showing the promise of efficient integration of transaction data into cryptocurrency price prediction. Our contributions can be summarized as:

- Our proposed motif convolution module-based graph representation learning approach combines graph neural networks to extract vertices temporally-evolving low-dimensional representations.
- To our best knowledge, our novel approach is the first method to systematically use attributed transaction graphs in cryptocurrency price prediction.
- Our approach outperforms all baselines which show the importance of efficient integration of transaction data into cryptocurrency price prediction.

1.1 Related Work

Lately, Bitcoin dynamics have been studied in terms of Bitcoin graph's attributes, as well as transaction cost. In this case, difficulty and hash rate have been identified as significantly correlated with Bitcoin price (Saad & Mohaisen, 2018). By utilizing these correlations, they develop a machine learning technique to increase Bitcoin price prediction accuracy. In another study, Jang and Lee (2018) has found core attributes of blockchain graphs to be predictive of highly volatile Bitcoin prices in addition to the exchange rates among major currencies. Since Bitcoin and Ethereum are decentralized cryptocurrencies, their network activity data such as transactions, are kept on blockchain. This storage puts forward the importance of on-chain data transactions and associated metrics to obtain practical knowledge of the network.

To analyze this chain transaction knowledge, we propose a method involving Graph Representation Learning and Graph Neural Networks (GNNs). Earlier network embedding approaches utilize biased random walks to conserve vertices local neighborhoods (Grover & Leskovec, 2016; Perozzi et al., 2014; Tang et al., 2015). Another line of works (Patro et al., 2012; Peng et al., 2020; Sefer & Kingsford, 2011; Sun et al., 2019; Veličković et al., 2019) maximizes the mutual information between global and local embeddings by training vertex encoders. Those approaches focus on preserving the node closeness where closer vertices will have similar embeddings. Nevertheless, those methods overemphasize node closeness at global structural knowledge's expense, so their performance decreases when making estimations at the graph level instead of the vertex level. As an example, earlier random walk-based approaches inspect only a restricted set of substructures such as subtrees as graph's representation. On the other hand, the most similar graph representation learning-based approach to ours is Wang et al. (2023), which also focuses on graph representation learning on the attributed relational graphs via motifs. Other similar methods on relational graphs such as Zhou et al. (2024) and Xu et al. (2024) have been suggested as well.

To pretrain GNNs, self-supervised techniques have been designed (Hu et al., 2020; Xu et al., 2021). Semantic and structural characterizations of graphs are learned via self-supervised training at graph, edge, and node levels which could be further finetuned for downstream tasks. Nevertheless, those methods could not extract substantial knowledge in graph motifs or subgraphs. Only a few methods have recently been proposed to handle knowledge at motif level. For instance, earlier studies Henderson et al. (2012) and Narayanan et al. (2016) focus on encoding local structures as binary attributes, which cannot take local structure deformations into account. There is still considerable room for improvement to benefit from local structural knowledge and generate vertex representations with high explainability. Efficient measurement of similarities among input graphs and high-quality motif dictionary's automatic generation appear as the top challenges of motif-based techniques.

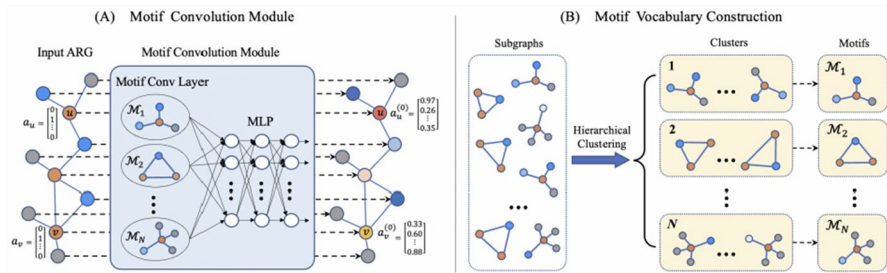


Fig. 1 Motif convolution module and motif dictionary construction. **A** The convolution operation computes the structural similarity score between each N motif and the subgraph centered at every vertex in the input graph to generate an N -dimensional context-sensitive embedding for the corresponding vertex. Such embedding is additionally altered via a multilayer perceptron neural network to generate an MCM embedding for the input vertex. For instance, even though 2 input vertices, u and v , characterize the identical elements (wallet in our transaction graphs), their corresponding MCM embeddings $a_u^{(0)}$ and $a_v^{(0)}$ are not identical since u and v are contextually different. Figure 2 illustrates MCM in detail. MCM's output could be an input into GNNs. **B** The motif dictionary is constructed by clustering over subgraphs sampled from input graphs

2 Motif-Based Graph Representation Learning

Our motif-based representation learning approach's main idea is motif convolution module (MCM) as seen in Fig. 1A. The motif convolution module incorporates a motif convolution layer (MCL) attached to an optional Multilayer Perceptron (MLP) neural network. The motifs in MCL represent spatial patterns, and they could be established via clustering subgraphs inferred over sampled graphs as seen in Fig. 1B. Those motifs could express different spatial structures that represent distinct local contexts. The convolution part produces local context-sensitive vertex representations that are transformation-invariant by applying all those motifs to every vertex in the input graph. MLP part could embed these vertex representations via further exploration of the interactions among motifs. MCM-generated vertex embeddings model context-aware local structures, that could enhance the capability of the following tasks in order to grasp rich semantic knowledge. Next, we thoroughly express motif dictionary generation, motif convolution, and integrating MCM with GNNs.

2.1 Motif Dictionary Construction

The ideal way to learn motif dictionaries is end-to-end, but such learning will be computationally expensive as seen in distinct domains (Besta et al., 2022; Chu & Stormo, 2022; Vincent-Cuaz et al., 2021). So, we instead used a simpler approach to build a motif dictionary that incorporates frequently occurring patterns in input ARGs. Firstly, a huge number of subgraphs, such as k -hop neighborhoods, are sampled over the dataset. A k -hop neighborhood (Nikolentzos et al., 2020) in graph theory refers to the set of nodes that can be reached from a specific node within k hops or k edges. In other words, it includes all nodes that are at most k edges away from the starting node. In this case, each sampled

subgraph keeps its center vertex. We ensure each sampled subgraph encloses the local context to the fullest extent by decreasing the subgraph sampling probability by half if the subgraph's center vertex is part of a sampled subgraph. Via such modification, we tend to sample unexplored local contexts with a higher probability. In this case, a single motif may represent subgraphs that are quite alike. We achieve such representation by grouping the sampled subgraphs into a predetermined number of clusters via hierarchical clustering with average linkage (Johnson, 1967). Afterwards, we select an exemplary subgraph for each cluster as a motif. When the overall subgraph set's size is exceedingly large for the clustering method, we randomly split the overall subgraph set into a number of smaller subsets, and infer a representative subgraph for each subset by applying the process above. Then, we apply this process to all representative subgraphs to extract the resulting motifs. Given each subgraph is modeled as ARG, we require pairwise similarities to apply hierarchical clustering between subgraphs.

2.2 Measuring Graph Similarity

As part of motif dictionary construction and motif convolution application, the similarity between 2 ARGs should be measured. We require such similarity metric not to be impacted by vertex permutation, so we need to match vertex-to-vertex between 2 graphs. Moreover, our similarity metric should not be impacted by graph sizes as we do not want a large graph to have a higher probability of being similar to a motif than a small graph. Let M represent a matching matrix for vertex-to-vertex matching between 2 ARGs G_1 and G_2 . Each matching matrix entry $M_{ui} \in \{0, 1\}$ defines whether vertex u in G_1 matches with vertex i in G_2 . By following the approach in Gold and Rangarajan (1996) and Menke and Yang (2020), normalized similarity between G_1 and G_2 is defined as:

$$S(G_1, G_2) = \left(\sum_{u=1}^{n_1} \sum_{i=1}^{n_2} \sum_{v=1}^{n_1} \sum_{j=1}^{n_2} \frac{M_{ui} M_{vj} s_1(e_{uv}^{(1)}, e_{ij}^{(2)})}{2\sqrt{l_1 \times l_2}} + \alpha S_\alpha \right) \times \frac{1}{1 + \alpha} \quad (1)$$

where $S_\alpha = \sum_{u=1}^{n_1} \sum_{i=1}^{n_2} \frac{M_{ui} s_2(u, i)}{\sqrt{n_1 \times n_2}}$, n_1 and n_2 are the vertex counts in G_1 and G_2 , respectively. l_1 and l_2 are the edge numbers in G_1 and G_2 , respectively. Edge agreement function $s_1(e_{uv}^1, e_{ij}^2)$ measures the similarity between $e_{uv}^1 \in G_1$ and $e_{ij}^2 \in G_2$, whereas vertex agreement function $s_2(u, i)$ measures the similarity between vertex $u \in G_1$ and vertex $i \in G_2$. Lastly, trade-off parameter α balances the impact of vertex similarities and edge similarities. Computation of $S(G_1, G_2)$ requires a matching matrix M . Matching 2 attributed relational graphs is a computationally difficult problem, and it has extensively been researched on Li et al. (2019) and Zager and Verghese (2008). In the following subsections, due to this computational complexity, we will analyze a cost-effective approach to estimate a suboptimal M and a GPU-fastened matching approach.

2.2.1 MCM's Graph Matching

The structural similarity value between the motif M_i from Motif Convolutional Layer and local substructure of vertex u from the input graph is calculated by convolution operation. Before convolution, optimal matching between M_i and u 's local subgraphs should be known. In the next subsection, we concisely present an NP-hard graph matching problem and discuss efficient approximation algorithms introduced by Gold and Rangarajan (1996) and Menke and Yang (2020). We also propose a GPU-enabled approach to fasten the matching in order to graph matching faster in practice and to achieve high-frequency calculations in MCM.

2.2.2 ARG Matching

Matching two ARGs optimally is computationally hard, and it can be expressed as a Quadratic Assignment Problem (QAP) Lawler (1963). In this case, we can solve the optimization problem below to match optimally:

$$\begin{aligned} \max_{M \in R^{n_1 \times n_2}} & \frac{1}{2} \sum_{u=1}^{n_1} \sum_{i=1}^{n_2} \sum_{v=1}^{n_1} \sum_{j=1}^{n_2} M_{ui} M_{vj} s_1(e_{uv}^{(1)}, e_{ij}^{(2)}) + \alpha \sum_{u=1}^{n_1} \sum_{i=1}^{n_2} M_{ui} s_2(u, i) \\ \text{s.t. } & \forall u \sum_{i=1}^{n_2} M_{ui} \leq 1, \forall i \sum_{u=1}^{n_1} M_{ui} \leq 1, \forall u, i M_{ui} \in \{0, 1\} \end{aligned} \quad (2)$$

where $s_1(e_{uv}, e_{ij})$, $s_2(u, i)$, α are already defined as part of Eq. 1. Gold and Rangarajan (1996) proposes a graduated assignment-based approach to find suboptimal matching between two attributed graphs. Menke and Yang (2020) has come up with a simpler and significantly faster version of this approach, without compromising accuracy too much. However, the resulting matching matrix from Menke and Yang (2020) may end up being ambiguous since it may not satisfy constraints at all times in Eq. 2. To ensure constraint satisfaction, we come up with an iterative greedy approach to modify the soft matching matrix M into a hard matching matrix which contains only binary values. Our greedy approach replaces the maximum value in M with 1, and replaces all remaining values in the identical row and column to 0. We keep applying this step iteratively to the remaining parts of M till each row/column sum is at most 1. In our large transaction graphs, this suboptimal greedy algorithm is still costly. As a result, we also implement a GPU-accelerated version.

2.2.3 Simpler and Faster Graduated Assignment Approach for Graph Matching

The original graduated assignment approach matches graphs suboptimally via solving the corresponding QAP's first-order Taylor expansion iteratively (Gold & Rangarajan, 1996). Afterwards, Menke and Yang (2020) has proposed a simpler and faster-graduated assignment approach, which pseudocode can be seen in Algorithm 1. After extracting the soft assignment matrix by relaxing the constraint $M_{ai} \in \{0, 1\}$ to a continuous range $[0, 1]$, this modified algorithm greedily turns the

soft assignment into a hard one. Soft assignment matrix can be obtained via Algorithm 1's iterative steps. Given the assignment matrix's initialization M^0 , we approximate the objective function $E(M)$ in Eq. 2 via its Taylor expansion at M^0 . After such expansion, the original optimization problem becomes identical to the assignment problem which instead maximizes $\sum_{a=1}^{n_1} \sum_{i=1}^{n_2} Q_{ai} M_{ai}$ where $Q_{ai} = \frac{\partial E(M)}{\partial M_{ai}}|_{M=M^0}$ represents the partial derivative. At the current step, the optimal M will be assigned back as the new initialization, and the Taylor expansion part will be repeated until convergence.

Algorithm 1 Simpler and Faster Graduated Assignment Approach for ARG Matching

```

1: Input:  $G_1, G_2, \beta_0, \beta_r, \beta_f$ 
2: Output: Hard assignment matrix  $M^*$ 
3:  $\beta = \beta_0$  ▷ Initialize  $\beta$ 
4:  $M_{ui} = s_1(u, i), \forall u \in G_1, \forall i \in G_2$  ▷ Initialize  $M$ 
5: while  $\beta \leq \beta_f$  do
6:    $\forall u \in G_1, \forall i \in G_2$ 
7:    $Q_{ui} = \frac{1}{2} \sum_{v=1}^{n_1} \sum_{j=1}^{n_2} s_2(e_{uv}, e_{ij}) M_{vj} + \alpha s_1(u, i)$ 
8:    $M_{ui} = \exp(\beta Q_{ui})$ 
9:    $\forall u \in G_1, \forall i \in G_2$ 
10:   $M'_{ui} = \frac{M_{ui}}{\sum_{j=1}^{n_2} M_{uj}}$ 
11:   $M_{ui} = \frac{M'_{ui}}{\sum_{v=1}^{n_1} M'_{vi}}$ 
12:   $\beta = \beta(1 + \beta_r)$ 
13: end while
14: return  $M^* \leftarrow \text{greedy\_hard\_assignment}(M)$ 

```

We take softmax with control parameter $\beta > 0$ across the constrained rows and columns to efficiently solve a constrained assignment where each row or column adds up to 1. Mathematically, $M = \text{softmax}(\beta Q)$, and higher β will eventually end up with a hard matching when $\beta \rightarrow \infty$ by pushing M 's entries to become either 0 or 1. In our assignment problem, graph matching includes 2 constraints, where both row and column should add up to 1. We satisfy both constraints as a result, by initially performing the element-wise exponential operation such that $M_{ai} = \exp(\beta Q_{ai})$. Afterwards, we normalize rows and columns in an alternating way until we converge to a doubly stochastic matrix which represents soft assignment between 2 input graphs (Sinkhorn, 1964). First, β is initialized as β_0 , and it is increased in β_r increments at each step till β hits the threshold β_f . Lastly, we convert soft M into hard assignment via the greedy approach.

2.2.4 GPU-Accelerated ARG Matching

We obtain accelerated matching by parallelizing the matching on GPUs. Line 5–13 of Algorithm 1 is implemented with a CUDA kernel for custom parallel processing of the multiple graph pairs matching. In particular, GPU's each cooperative thread

array (CTA) is used in matching computation between two graphs. In Algorithm 1, partial derivative, and exponential computations require element-wise operations to be performed. As a result, each thread inside CTA is used in computing the assignment matrix's single element, and the assignment matrix is normalized by all threads working in cooperation. Such cooperation benefits from different degrees of parallelism on the GPU. CUDA kernels are also implemented to compute the remaining vertex and edge similarity functions as well as the greedy hard assignment calculation.

2.3 Motif Convolution

The similarity score between each motif and the subgraph centered at input graph's each vertex is calculated by the motif convolution layer. We obtain each input vertex's motif representation by appending the similarity scores between the subgraph of the vertex and all motifs. Such representation could be an input for a trainable MLP layer with non-linear activation functions such as ReLU. Feeding the representation into such a layer will generate an enhanced embedding which can encode interactions between motif attributes as well. This operation can be denoted as:

$$a_u^{(0)} = MCM(u \in G; \{M_i\}_{i=0}^N) \quad (3)$$

where G is an input ARG, u is a vertex in G , $\{M_i\}_{i=0}^N$ defines the motif dictionary of size N , and $a_u^{(0)}$ represents u 's MCM embedding. MCM computation is illustrated in detail in Fig. 2. For each vertex in the input graph, the similarities between the whole set of motifs and the vertex's local structure are appended to generate an N -dimensional context-sensitive embedding, on which the local structural attributes characterized via motifs are encoded. We can further transform the motif attribute representations via a trainable MLP neural network in order to generate the ultimate MCM embedding for the input vertex. If one removes the MLP part, motif attribute

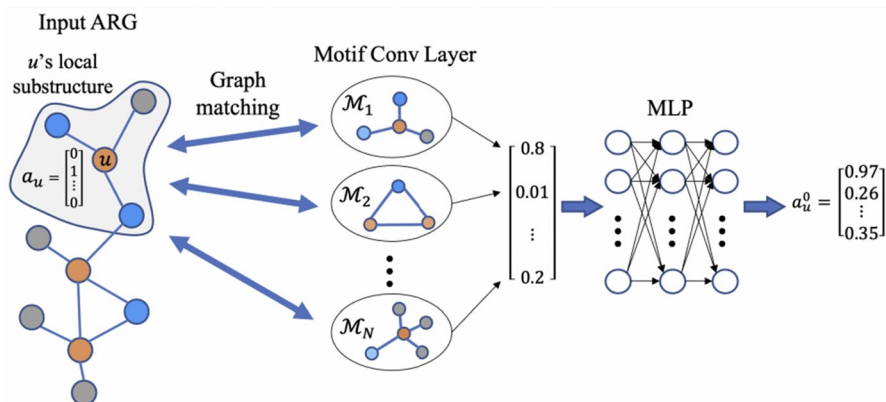


Fig. 2 Motif convolution module. Here, the convolution operation matches every motif with the local structure centering at every vertex in the input graph

representations will directly become the MCM embedding for the input vertex. Here, we obtain motifs by a pretraining procedure discussed previously in Sect. 2.1. The MLP component must be trained with the downstream task.

2.4 Integrating Motif Convolutions with GATs

The MCM could be seen as the initial component for any GNN while forming MCM+GNN. MCM component's output is an attributed relational graph, which could be input into any GNN accepting an ARG as input. Here, we use Graph Attention Network (GAT) (Velickovic et al., 2017). GATs incorporate attention mechanisms, allowing the model to focus on the most relevant parts of a graph when making predictions. GATs compute attention scores between a node and its neighbors to determine the importance of each neighbor's contribution. More formally, let $\mathcal{G} = (V, E)$ be the output of MCM component with nodes V and edges E , $h_i \in \mathbb{R}^F$ be feature vector of node i , and $N(i)$ be set of neighbors of node i (including i itself). The attention coefficient between nodes i and j is computed as:

$$e_{ij} = a(Wh_i, Wh_j) \quad (4)$$

where $W \in \mathbb{R}^{F' \times F}$ is weight matrix for feature transformation and $a : \mathbb{R}^{2F'} \rightarrow \mathbb{R}$ is learnable attention function. In practice, a often uses a shared weight vector $\vec{a}$ and applies a non-linear function like LeakyReLU:

$$e_{ij} = \text{LeakyReLU}(\vec{a}^T [Wh_i \| Wh_j]) \quad (5)$$

We normalize attention scores using softmax:

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N(i)} \exp(e_{ik})} \quad (6)$$

where the updated feature vector for node i is computed as:

$$h'_i = \sigma \left(\sum_{j \in N(i)} \alpha_{ij} Wh_j \right) \quad (7)$$

To stabilize learning, multiple attention heads are used. If there are K heads, their outputs can be concatenated:

$$h'_i = \parallel_{k=1}^K \sigma \left(\sum_{j \in N(i)} \alpha_{ij}^{(k)} W^{(k)} h_j \right) \quad (8)$$

In our case, we feed the ultimate representation h'_i into a trainable part such as a fully connected layer as part of predicting prices.

2.5 Mathematical Definition of Backpropagation in GNNs

Given a graph $G = (V, E)$, let $\mathbf{h}_v^{(l)}$ be the feature vector of node v at layer l , $\mathcal{N}(v)$ be the set of neighbors of node v , and $\mathbf{W}^{(l)}, \mathbf{b}^{(l)}$ be learnable weights and biases. The forward pass updates node features using an aggregation and transformation function:

$$\mathbf{h}_v^{(l+1)} = \sigma(\mathbf{W}^{(l)} \cdot \text{AGGREGATE}(\{\mathbf{h}_u^{(l)} : u \in \mathcal{N}(v)\}) + \mathbf{b}^{(l)})$$

where σ is a nonlinear activation function and AGGREGATE can be sum, mean, max-pooling, or attention-based mechanisms. Using the chain rule, the gradients are computed as follows:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_v^{(L)}} = \frac{\partial \mathcal{L}(\hat{y}_v, y_v)}{\partial \hat{y}_v} \cdot \frac{\partial \hat{y}_v}{\partial \mathbf{h}_v^{(L)}}$$

For layer l , we propagate the error back:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{h}_v^{(l)}} = \sum_{u \in \mathcal{N}(v)} \frac{\partial \mathcal{L}}{\partial \mathbf{h}_u^{(l+1)}} \cdot \frac{\partial \mathbf{h}_u^{(l+1)}}{\partial \mathbf{h}_v^{(l)}}$$

Using gradient descent:

$$\begin{aligned} \mathbf{W}^{(l)} &\leftarrow \mathbf{W}^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} \\ \mathbf{b}^{(l)} &\leftarrow \mathbf{b}^{(l)} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} \end{aligned}$$

where η is the learning rate.

3 GNN + Self Attention-Based LSTM for Price Prediction

Once graph representations are learned for different time steps, such representation vector and historical price become input to multivariate self-attention-based LSTM (Hochreiter & Schmidhuber, 1997). The prediction process is summarized in Fig. 3. LSTM is frequently utilized in sequence-based modeling tasks, such as time series forecasting, where it can use the data arranged in a consecutive order. In this case, LSTM works in extracting the primary attributes which directly inputs time series data and infers significant temporal knowledge. Latent states are updated recurrently via LSTM by integrating the previous time step's output as an existing input while learning the sequential nature of time series data. Moreover, LSTM uses a cell state, which captures longer dependencies over sequences as well as prevents gradient vanishing and explosion problems. In addition to the cell state which models the memory behaviour, there are 3 more gates in LSTM to forget and update the flowing knowledge.

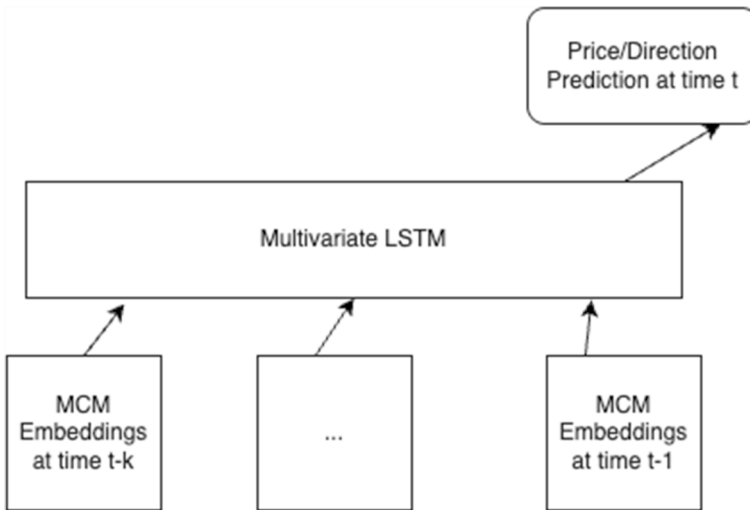


Fig. 3 Summary of LSTM on top of MCM embeddings obtained at distinct time steps

3.1 Attention Mechanism

The attention mechanism focuses on helping a model in learning the position to attend as part of training, and it is an extra part of which could be utilized inside a neural network architecture (Bahdanau et al., 2015).

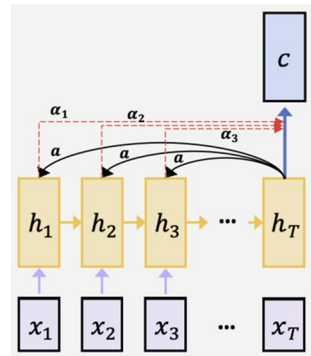
Lately, the attention mechanism has been used in many sequence-based modeling methods such as BERT, and transformer, while modeling sequences for machine translation and classifying documents (Devlin et al., 2019; Vaswani et al., 2017). The attention mechanism is particularly practical in scenarios where the model should be selective in exploiting important parts of a large knowledge base. A number of different attention mechanisms exist where they vary according to their internal computations. Since LSTM in this study utilizes multivariate time series data for forecasts, we attach the attention mechanism to LSTM unit's end. We employ a self-attention, namely Bahdanau attention, in our proposed model as shown in Fig. 4. This attention mechanism's computation can be expressed as:

$$e_j = a(h_T, h_j) \quad (9)$$

$$a_j = \frac{\exp(e_j)}{\sum_{k=1}^T \exp(e_k)} \quad (10)$$

$$c = \sum_{j=1}^T a_j h_j \quad (11)$$

Fig. 4 Attention mechanism's illustration



where j is timestamp inside time series data for $j = 1, \dots, T$ and a is an alignment unit which is modeled as a feedforward neural network using a hyperbolic tangent activation function.

4 Experimental Setup

4.1 Datasets

We have obtained Bitcoin and Ethereum transaction datasets from Bitcoin's and Ethereum's public blockchains respectively, by using publicly available APIs from 2016 until the end of 2024 (Motamed & Bahrak, 2019). This period is quite comprehensive as it includes multiple bull and bear market periods as well as the recent COVID-19 pandemic period. Even though the blockchain structures for Bitcoin and Ethereum are quite different, the transaction knowledge is kept inside the blocks in both of them. A predetermined number of transactions could be contained inside each block.

Generally, we can divide blockchains into 2 different types such as account-based and UTXO (Unspent Transaction Output)-based. Bitcoin uses a UTXO-based blockchain, where every individual transaction input is connected to a previous transactions output. To put it in another way, the existing transactions will utilize the previous transactions outputs and will generate new outputs to be utilized in the following transactions. However, Ethereum is an account-based blockchain, where we explicitly state the addresses of the incoming and outgoing accounts explicitly. In Bitcoin, every observed transaction includes a number of inputs and outputs, whereas Ethereum's each transaction includes a single input and output. In our experiments, over a transaction graph, vertices correspond to account addresses, and edges correspond to the transactions between those accounts. Since blockchains each recorded block owns a timestamp, we have partitioned our timeline into weekly intervals and generated a transaction graph each week which includes the corresponding block's transactions. We also obtain the daily Bitcoin and Ethereum prices from Yahoo Finance from 2016 until the end of

2024. We run our experiments on Google Colab with Nvidia P100 with 52 GB memory. On average, Bitcoin graphs density is 0.002 whereas it is 0.003 for Ethereum.

4.2 Evaluation

Since we have carried out a performance evaluation over the trained models, we have selected the test dataset in a continuous interval. In our case, we have utilized a sliding window-based technique where we keep training on consecutive 2-year intervals and predict over the subsequent 1-year timelines. In this sliding window with a retraining-based approach, we keep shifting the train and test set periods by a single year forward and repeat this training and testing process. As a result, each year between 2016 and 2024 are tested once by our retraining technique.

In terms of performance evaluation for price prediction, we choose several commonly used regression evaluation metrics for experimental results validation. More specifically, we use Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE). Those metrics are expressed explicitly as follows:

$$MAE = \frac{1}{N} \sum_{t=1}^N |x_t - \hat{x}_t| \quad (12)$$

$$MSE = \frac{1}{N} \sum_{t=1}^N (x_t - \hat{x}_t)^2 \quad (13)$$

$$RMSE = \sqrt{\frac{1}{N} \sum_{t=1}^N (x_t - \hat{x}_t)^2} \quad (14)$$

$$MAPE = \frac{100}{N} \sum_{t=1}^N \frac{|x_t - \hat{x}_t|}{x_t} \quad (15)$$

where x_t is the actual price at time t , $\hat{x}_t$ is the predicted price at time t , and N represents the number of prediction points.

We also use our method to predict price directions, whether price will increase or decrease. In this case, we use the same methods except the last layer becomes a softmax layer to handle classification. In terms of price direction prediction, we use the following classification metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (16)$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (17)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (18)$$

$$\text{F1 Score} = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (19)$$

where TP, TN, FP, and FN mean true positive, true negative, false positive and false negative sample counts, respectively.

5 Results

5.1 Price and Price Direction Prediction Performance Relative to the Baselines

We have evaluated the performance of our motif-based GNN + LSTM method with respect to 4 machine learning baseline approaches which only use historical price data, such as SVM (Cortes, 1995), LSTM (Alpaydin, 2014), HLSTM (Hou et al., 2020), and GRU (Cho, 2014). According to Table 1, among the baselines, LSTM which uses only price data appears to perform better than SVM which uses only price data according to almost all considered metrics for both Bitcoin and Ethereum price prediction. Among other competing techniques, GRU slightly outperforms LSTM whereas hierarchical LSTM (HLSTM) generally performs similar to LSTM. Both of these baselines which cannot utilize transaction graph structure are remarkably outperformed by our approach which can utilize both blockchain transaction knowledge and historical prices. However, in general, the predictability of the Ethereum price appears to be slightly better than the one for Bitcoin. These results clearly demonstrate the importance of integrating the transaction graph knowledge into price prediction, as well as the quality of GNNs in embedding and extracting such extra knowledge.

Table 1 The performance comparison of different methods for price prediction for Bitcoin and Ethereum in terms of various regression metrics

Crypto	Methods	MAE	MSE	RMSE	MAPE
Bitcoin	GNN + LSTM (Proposed)	26.41	936.21	30.59	13.62
	Only price via LSTM	32.21	1035.22	32.17	21.12
	Only price via hierarchical LSTM	31.89	1025.52	32.02	17.85
	Only price via GRU	30.25	984.45	31.37	16.99
	Only price via SVM	34.15	1155.81	33.99	23.25
Ethereum	GNN + LSTM (Proposed)	25.21	876.54	29.60	12.21
	Only price via LSTM	30.99	1002.21	31.65	14.85
	Only price via hierarchical LSTM	29.99	984.45	31.37	14.65
	Only price via GRU	28.87	960.23	30.98	13.89
	Only price via SVM	32.87	1104.43	33.23	19.21

The best score is highlighted in bold for each metric

Similarly, we have evaluated the performance of our motif-based GNN + LSTM method with respect to the same baselines above in terms of price direction prediction. According to Table 2, similar to Table 1, LSTM baseline that uses only price data appears to perform better than SVM baseline which uses only price data according to almost all considered metrics for both Bitcoin and Ethereum price direction prediction. Both of these baselines which cannot utilize transaction graph structure are remarkably outperformed by our approach which can utilize both blockchain transaction knowledge and historical prices. Similar to Table 1, these results clearly demonstrate the importance of integrating the transaction graph knowledge into price direction prediction as well.

5.2 The Impact of Different GNN Types

We have also assessed the impact of using different types of GNNs while obtaining the graph representations. In this case, we have compared the performance of embedding the transaction network via our motif-based approach combined with GAT. Comparison is made with respect to 4 other well-known GNNs in the literature: Node2Vec (Grover & Leskovec, 2016), GraphWave (Donnat et al., 2018), Graph Isomorphism Network (GIN) (Xu et al., 2018), and WatchYourStep (Abu-El-Haija et al., 2018). Among these GNNs, Node2Vec incorporates multiple distinct descriptions for vertex neighborhoods in networks via biased random walk simulation. Unlike the regular random walk, in the Node2Vec algorithm, the nodes that are visited before affect the possibility of the nodes that will be visited. Another scalable method GraphWave is an unsupervised learning-based approach to infer vertex representations by considering the structural similarity in graphs. GraphWave is based on using spectral graph wavelets, where wavelets are treated as probability distributions. GraphWave is based on embedding those wavelet distributions via empirical characteristic functions, while providing vector embeddings that could be treated as input to a machine learning algorithm. Graph Isomorphism Networks (GINs) Xu

Table 2 The performance comparison of different methods for price direction prediction for Bitcoin and Ethereum in terms of various classification metrics

Crypto	Methods	Accuracy	Precision	Recall	F1 Score
Bitcoin	GNN + LSTM (Proposed)	0.765	0.763	0.753	0.758
	Only price via LSTM	0.715	0.721	0.713	0.717
	Only price via hierarchical LSTM	0.725	0.720	0.725	0.722
	Only price via GRU	0.728	0.740	0.745	0.742
	Only price via SVM	0.652	0.651	0.661	0.656
Ethereum	GNN + LSTM (Proposed)	0.804	0.762	0.748	0.755
	Only price via LSTM	0.722	0.723	0.705	0.714
	Only price via hierarchical LSTM	0.745	0.735	0.745	0.740
	Only price via GRU	0.765	0.740	0.750	0.745
	Only price via SVM	0.661	0.655	0.675	0.665

The best score is highlighted in bold for each metric

et al. (2018) are a type of Graph Neural Network designed to achieve strong discriminative power for graph-structured data. GINs were proposed to address the expressiveness limitations of earlier GNNs like Graph Convolutional Networks (GCNs) (Kipf & Welling, 2016). They can distinguish graphs based on their structure and are as powerful as the Weisfeiler-Lehman (WL) graph isomorphism test, a classical method for testing graph similarity. Lastly, in the WatchYourStep algorithm, trainable hyperparameters replace the fixed hyperparameters where trainable ones can be automatically estimated by backpropagation. Different than the remaining techniques, WatchYourStep integrates the attention parameters into the random walk traversal.

According to Table 3, our motif-based graph embedding approach performs better than the other GNNs in most metrics. Among the existing GNNs, GraphWave and GIN perform better than both Node2Vec and WatchYourStep for both Bitcoin and Ethereum price direction prediction. These results also show the importance of GNN-based embeddings of transaction graphs in cryptoasset price prediction.

5.3 The Impact of Whale Elimination Before Graph Embedding

As crypto-whales influence the market volatility (Herremans & Low, 2022), the effect of disregarding whale transactions in our prediction models is also tested in this study. To detect whales, firstly, the maximum amount of total weekly transaction volume is found. Then, a whale threshold is defined as 90% of the maximum. Finally, wallets that have more than the whale threshold are marked as whales, and their transactions are dropped from the transaction graph. To test whether the whale transactions distort the data, whale-eliminated and original transaction datasets are used together for comparison. According to Table 4, whale elimination slightly enhances the prediction performance for both Bitcoin and Ethereum in terms of price prediction.

Table 3 The performance comparison of different Graph Neural Networks for price direction prediction for Bitcoin and Ethereum in terms of various classification metrics: Accuracy, Precision, Recall, and F1 Score

Crypto	Methods	Accuracy	Precision	Recall	F1 Score
Bitcoin	Motif-based GNN + LSTM (Proposed)	0.765	0.763	0.753	0.758
	Node2Vec + LSTM	0.722	0.732	0.744	0.738
	GraphWave + LSTM	0.767	0.751	0.741	0.746
	GIN + LSTM	0.777	0.752	0.744	0.748
	WatchYourStep + LSTM	0.731	0.735	0.755	0.745
Ethereum	Motif-based GNN + LSTM (Proposed)	0.804	0.762	0.748	0.755
	Node2Vec + LSTM	0.772	0.750	0.730	0.740
	GraphWave + LSTM	0.802	0.752	0.742	0.747
	GIN + LSTM	0.812	0.754	0.744	0.749
	WatchYourStep + LSTM	0.775	0.755	0.745	0.750

The best score is highlighted in bold for each metric

Table 4 The impact of eliminating the whales before embedding via GNN for Bitcoin and Ethereum price prediction

Crypto	Methods	MAE	MSE	RMSE	MAPE
Bitcoin	GNN + LSTM (Proposed)	26.41	936.21	30.59	13.62
	GNN + LSTM + Whale elimination	24.91	888.13	29.15	13.88
Ethereum	GNN + LSTM (Proposed)	25.21	876.54	29.60	12.21
	GNN + LSTM + Whale elimination	24.99	865.12	28.11	12.13

The highest score is highlighted in bold

5.4 The Impact of The Interactions Between Funds

We also focused on a subset of Ethereum accounts labeled on Etherscan as likely trading with decentralized finance (DeFi) protocols. We obtained the list of all “fund” accounts from Etherscan, which consists of 65 entities. Among these, 25 are identified as associated with Alameda Research, a cryptocurrency trading firm linked to FTX that was active in DeFi before the collapse of both entities in November 2022 due to massive fraud. This subset was chosen to examine the DeFi interactions of notable trading accounts rather than to analyze the entire blockchain. Although this list does not cover all accounts, it provides a focused perspective on significant players in the DeFi space

According to Table 5, our motif-based GNN + LSTM method outperforms the rest of the baselines. Among the baselines, LSTM which uses only price data appears to perform better than SVM which uses only price data according to almost all considered metrics for both Bitcoin and Ethereum price prediction. Among other competing techniques, GRU slightly outperforms LSTM whereas Hierarchical LSTM (HLSTM) generally performs similar to LSTM. Both of these baselines which cannot utilize transaction graph structure are remarkably outperformed by our approach which can utilize both blockchain transaction knowledge and historical prices. Similar to our results on the whole interaction networks, these results on a subset of the fund interaction graphs clearly demonstrate the importance of integrating the transaction graph knowledge into price prediction, as well as the quality of GNNs in embedding and extracting such extra knowledge.

Table 5 The performance comparison of different methods for price prediction for Ethereum in terms of various regression metrics

Methods	MAE	MSE	RMSE	MAPE
GNN + LSTM (Proposed)	7.23	176.54	13.28	6.21
Only price via LSTM	10.21	202.21	14.22	7.28
Only price via hierarchical LSTM	9.12	197.75	14.06	7.12
Only price via GRU	8.11	190.12	13.78	6.55
Only price via SVM	10.45	204.43	14.29	7.99

The best score is highlighted in bold for each metric

Table 6 The performance comparison of different methods for price direction prediction for Ethereum in terms of various classification metrics

Methods	Accuracy	Precision	Recall	F1 Score
GNN + LSTM (Proposed)	0.882	0.815	0.778	0.796
Only price via LSTM	0.845	0.785	0.745	0.764
Only price via hierarchical LSTM	0.862	0.795	0.755	0.774
Only price via GRU	0.870	0.801	0.763	0.781
Only price via SVM	0.821	0.762	0.722	0.742

The best score is highlighted in bold for each metric

In terms of price direction prediction, our motif-based GNN + LSTM approach again outperforms all competing approaches as seen in Table 6. Similar to Table 5, these results clearly demonstrate the importance of integrating the transaction graph knowledge of funds into price direction prediction as well.

6 Conclusions

The authors came up with a new technique which combines Graph Neural Networks with LSTM to combine historical prices with blockchain transaction graph-based attributes to forecast Bitcoin's and Ethereum's future prices via deep learning methods. More specifically, we came up with Motif Convolution Module (MCM), a motif-based graph representation learning approach to take local structural knowledge into account more strongly in node and edge-attributed transaction graphs encoding substantial structural knowledge. Our proposed MCM constructs a motif dictionary without supervision and employs a new motif convolution operation while extracting the vertices local structural context. Afterwards, we learn high-level vertex embeddings by using such structural context via multilayer perceptron and graph neural network. Overall, we extract the attributed transaction graphs temporally-evolving low-dimensional representations, and use such embedding data together with historical prices within LSTM to predict future prices accurately. Our proposed approach outperforms all considered baselines in terms of both price and price direction prediction, showing the promise of efficient integration of transaction data into cryptocurrency price prediction. Rigorous experiments also verify the robustness of our approach.

In future work, one viable study could be predicting the crypto asset prices and dynamics via a more comprehensive framework. More specifically, a comprehensive framework that combines different types of datasets such as social media data, Google Trends, and on-chain data could be developed.

Author Contributions PC: Methodology, Software, Visualization, Writing. ES: Conceptualization, Formal Analysis, Methodology, Writing. All authors read and approved the final manuscript.

Funding Open access funding provided by the Scientific and Technological Research Council of Türkiye (TÜBİTAK). This research was funded by TÜBİTAK (Scientific and Technological Research Council of Turkey) 3501 Project with grant number 122E706.

Data Availability The datasets used and/or analyzed during the current study are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Ethical Approval Not applicable.

Consent to Participate Not applicable.

Consent for Publication Not applicable.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Abu-El-Haija, S., Perozzi, B., Al-Rfou, R., & Alemi, A. (2018). Watch your step: Learning node embeddings via graph attention. In *Proceedings of the 32nd international conference on neural information processing systems, NIPS'18* (pp. 9198–9208). Curran Associates Inc.
- Alpaydm, E. (2014). *Introduction to machine learning. Adaptive computation and machine learning series* (3rd ed.). The MIT Press.
- Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In Y. Bengio & Y. LeCun (Eds.), *3rd international conference on learning representations, ICLR 2015. Conference track proceedings*, San Diego, CA, USA, May 7–9, 2015.
- Besta, M., Grob, R., Miglioli, C., Bernold, N., Kwasniewski, G., Gjini, G., Kanakagiri, R., Ashkboos, S., Gianinazzi, L., & Dryden, N. (2022). Motif prediction with graph neural networks. [arXiv:2106.00761](https://arxiv.org/abs/2106.00761)
- Cho, K. (2014). Learning phrase representations using rnn encoder-decoder for statistical machine translation. [arXiv:1406.1078](https://arxiv.org/abs/1406.1078)
- Chu, S., & Stormo, G. (2022). Deep unfolded convolutional dictionary learning for motif discovery. *bioRxiv* <https://doi.org/10.1101/2022.11.06.515322>. <https://www.biorxiv.org/content/early/2022/11/07/2022.11.06.515322>
- Cortes, C. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American Chapter of the Association for Computational Linguistics: Human language technologies, Volume 1 (Long and Short Papers)* (pp. 4171–4186). Association for Computational Linguistics.
- Donnat, C., Zitnik, M., Hallac, D., & Leskovec, J. (2018). Learning structural node embeddings via diffusion wavelets. In *International ACM conference on knowledge discovery and data mining (KDD)*.
- Gezici, A. H. B., & Sefer, E. (2024). Deep transformer-based asset price and direction prediction. *IEEE Access*, 12, 24164–24178. <https://doi.org/10.1109/ACCESS.2024.3358452>

- Gold, S., & Rangarajan, A. (1996). A graduated assignment algorithm for graph matching. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(4), 377–388.
- Greaves, A., & Au, B. (2015). Using the bitcoin transaction graph to predict the price of bitcoin. *No data*, 8, 416–443.
- Grover, A., & Leskovec, J. (2016). Node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining, KDD '16* (pp. 855–864). Association for Computing Machinery, New York, NY, USA.
- Henderson, K., Gallagher, B., Eliassi-Rad, T., Tong, H., Basu, S., Akoglu, L., Koutra, D., Faloutsos, C., & Li, L. (2012). Rolx: Structural role extraction & mining in large graphs. In *Proceedings of the 18th ACM SIGKDD international conference on knowledge discovery and data mining, KDD '12* (pp. 1231–1239). Association for Computing Machinery, New York, NY, USA.
- Herremans, D., & Low, K. W. (2022). Forecasting bitcoin volatility spikes from whale transactions and cryptoquant data using synthesizer transformer models. [arXiv:2211.08281](https://arxiv.org/abs/2211.08281)
- Herskind, L., Katsikouli, P., & Dragoni, N. (2020). Privacy and cryptocurrencies-a systematic literature review. *IEEE Access*, 8, 54044–54059.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Hou, H., Xu, Y., Chen, M., Liu, Z., Guo, W., Gao, M., Xin, Y., & Cui, L. (2020). Hierarchical long short-term memory network for cyberattack detection. *IEEE Access*, 8, 90907–90913. <https://doi.org/10.1109/ACCESS.2020.2983953>
- Hu, W., Liu, B., Gomes, J., Zitnik, M., Liang, P., Pande, V., & Leskovec, J. (2020). Strategies for pre-training graph neural networks. In *International conference on learning representations*.
- Huang, J. Z., Huang, W., & Ni, J. (2019). Predicting bitcoin returns using high-dimensional technical indicators. *The Journal of Finance and Data Science*, 5(3), 140–155.
- Huawei & Oxford Economics (2017). Digital spillover: Measuring the true impact of the digital economy.
- Indera, N., Yassin, I., Zabidi, A., & Rizman, Z. (2017). Non-linear autoregressive with exogenous input (narx) bitcoin price prediction model using pso and moving average technical indicators. *Journal of Fundamental and Applied Sciences*, 9, 791–808.
- Jagannath, N., Barbulescu, T., Sallam, K. M., Elgendi, I., Mcgrath, B., Jamalipour, A., Abdel-Basset, M., & Munasinghe, K. (2021a). An on-chain analysis-based approach to predict ethereum prices. *IEEE Access*, 9, 167972–167989.
- Jagannath, N., Barbulescu, T., Sallam, K. M., Elgendi, I., Okon, A. A., Mcgrath, B., Jamalipour, A., & Munasinghe, K. (2021b). A self-adaptive deep learning-based algorithm for predictive analysis of bitcoin price. *IEEE Access*, 9, 34054–34066.
- Jang, H., & Lee, J. (2018). An empirical study on modeling and prediction of bitcoin prices with bayesian neural networks based on blockchain information. *IEEE Access*, 6, 5427–5437.
- Jay, P., Kalariya, V., Parmar, P., Tanwar, S., Kumar, N., & Alazab, M. (2020). Stochastic neural networks for cryptocurrency price prediction. *IEEE Access*, 8, 82804–82818.
- Johnson, S. C. (1967). Hierarchical clustering schemes. *Psychometrika*, 32(3), 241–254.
- Kim, G., Shin, D. H., Choi, J. G., & Lim, S. (2022). A deep learning-based cryptocurrency price prediction model that uses on-chain data. *IEEE Access*, 10, 56232–56248.
- Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. [arXiv:1609.02907](https://arxiv.org/abs/1609.02907)
- Lawler, E. L. (1963). The quadratic assignment problem. *Management Science*, 9(4), 586–599.
- Li, Y., Gu, C., Dullien, T., Vinyals, O., & Kohli, P. (2019). Graph matching networks for learning the similarity of graph structured objects. [arXiv:1904.12787](https://arxiv.org/abs/1904.12787)
- Menke, J., & Yang, A. Y. (2020). Graduated assignment graph matching for realtime matching of image wireframes. In *2020 IEEE/RSJ international conference on intelligent robots and systems (IROS)* (pp. 5909–5916).
- Motamed, A. P., & Bahrak, B. (2019). Quantitative analysis of cryptocurrencies transaction graph. *Applied Network Science*, 4(1), 131.
- Mudassir, M., Bennbaia, S., Unal, D., & Hammoudeh, M. (2020). Time-series forecasting of bitcoin prices using high-dimensional features: A machine learning approach. *Neural Computing and Applications*, 1–15.
- Narayanan, A., Chandramohan, M., Chen, L., Liu, Y., & Saminathan, S. (2016). subgraph2vec: Learning distributed representations of rooted sub-graphs from large graphs. [arXiv:1606.08928](https://arxiv.org/abs/1606.08928)
- Nikolentzos, G., Dasoulas, G., & Vazirgiannis, M. (2020). K-hop graph neural networks. *Neural Networks*, 130, 195–205.

- Patro, R., Duggal, G., Sefer, E., Wang, H., Filippova, D., & Kingsford, C. (2012). The missing models: A data-driven approach for learning how networks grow. In *Proceedings of the 18th ACM SIGKDD international conference on knowledge discovery and data mining, KDD '12* (pp. 42–50). Association for Computing Machinery, New York, NY, USA. <https://doi.org/10.1145/2339530.2339541>
- Patro, R., Sefer, E., Malin, J., Marçais, G., Navlakha, S., & Kingsford, C. (2011). Parsimonious reconstruction of network evolution. In T. M. Przytycka & M. F. Sagot (Eds.), *Algorithms in bioinformatics* (pp. 237–249). Springer.
- Peng, Z., Huang, W., Luo, M., Zheng, Q., Rong, Y., Xu, T., & Huang, J. (2020). Graph representation learning via graphical mutual information maximization. In *Proceedings of the Web conference 2020, WWW '20* (pp. 259–270). Association for Computing Machinery, New York, NY, USA.
- Perozzi, B., Al-Rfou, R., & Skiena, S. (2014). Deepwalk: Online learning of social representations. In *Proceedings of the 20th ACM SIGKDD international conference on knowledge discovery and data mining, KDD '14* (pp. 701–710). Association for Computing Machinery, New York, NY, USA.
- Saad, M., & Mohaisen, A. (2018). Towards characterizing blockchain-based cryptocurrencies for highly-accurate predictions. In *IEEE INFOCOM 2018—IEEE conference on computer communications workshops (INFOCOM WKSHPS)* (pp. 704–709).
- Sefer, E. (2021). Hi-c interaction graph analysis reveals the impact of histone modifications in chromatin shape. *Applied Network Science*, 6(1), 54. <https://doi.org/10.1007/s41109-021-00396-1>
- Sefer, E. (2022a). Biocode: A data-driven procedure to learn the growth of biological networks. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, 19(6), 3103–3113.
- Sefer, E. (2022b). Probe: Joint modeling of epigenome and transcriptome effects in 3d genome. *BMC Genomics*, 23(1), 287.
- Sefer, E., & Kingsford, C. (2011). Metric labeling and semi-metric embedding for protein annotation prediction. In V. Bafna & S. C. Sahinalp (Eds.), *Research in computational molecular biology* (pp. 392–407). Springer.
- Sinkhorn, R. (1964). A relationship between arbitrary positive matrices and doubly stochastic matrices. *The Annals of Mathematical Statistics*, 35(2), 876–879.
- Sun, F. Y., Hoffman, J., Verma, V., & Tang, J. (2019). Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization. In *International conference on learning representations*.
- Tang, J., Qu, M., Wang, M., Zhang, M., Yan, J., & Mei, Q. (2015). Line: Large-scale information network embedding. In *Proceedings of the 24th international conference on World Wide Web, WWW '15* (pp. 1067–1077). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, et al. (Eds.), *Advances in neural information processing systems* (Vol. 30). Curran Associates Inc.
- Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. *Stat*, 1050(20), 10–48550.
- Veličković, P., Fedus, W., Hamilton, W. L., Liò, P., Bengio, Y., & Hjelm, R. D. (2019). Deep graph info-max. In *International conference on learning representations*.
- Vincent-Cuaz, C., Vayer, T., Flamar, R., Corneli, M., & Courty, N. (2021). Online graph dictionary learning. [arXiv:2102.06555](https://arxiv.org/abs/2102.06555)
- Wang, Y., Chen, S., Chen, G., Shurberg, E., Liu, H., & Hong, P. (2023). Motif-based graph representation learning with application to chemical molecules. *Informatics*, 10(1), 8.
- Xu, H., Zhou, Z., & Hong, P. (2024). Graph multi-similarity learning for molecular property prediction. [arXiv:2401.17615](https://arxiv.org/abs/2401.17615)
- Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2018). How powerful are graph neural networks? [arXiv:1810.00826](https://arxiv.org/abs/1810.00826)
- Xu, M., Wang, H., Ni, B., Guo, H., & Tang, J. (2021). Self-supervised graph-level representation learning with local and global structure. [arXiv:2106.04113](https://arxiv.org/abs/2106.04113)
- Zager, L. A., & Verghese, G. C. (2008). Graph similarity scoring and matching. *Applied Mathematics Letters*, 21(1), 86–94.
- Zhou, Z., Xu, H., & Hong, P. (2024). Multimodal fusion with relational learning for molecular property prediction. [arXiv:2410.12128](https://arxiv.org/abs/2410.12128)